

SQLite-Carver-Pro v1.7.0

Technical Reference Manual & Low-Level Forensic Engineering Specifications

 **Physical B-Tree Dissection, Slack Carving & Reverse Engineering Architecture**

Comprehensive technical manual detailing physical page layout mechanics, deleted cell reconstruction algorithms, per-page steganography detection, WAL transaction timeline diffing, and database schema resurrection without the SQLite SQL interpreter.

Author: Dam-FOR3K

Version: 1.7.0 (Professional Forensic Edition)

Target Architecture: Standalone Air-Gapped Engine (Windows x86_64, Linux, macOS)

Methodological Standard: ISO/IEC 27037 Digital Evidence Handling & Forensic Soundness

1. Physical SQLite Architecture & Forensic Audit Areas

Unlike standard database management suites that depend on the official SQLite engine library (`sqlite3.dll`), **SQLite-Carver-Pro** operates at the binary level directly on raw database disk pages. This low-level approach bypasses database engine abstraction layers, allowing the direct recovery of unallocated, overwritten, fragmented, and anti-forensically concealed data.

1.1 The 100-Byte Database Header

The first 100 bytes of a database file (located exclusively on Page 1) define the global database parameters. Each field holds critical forensic value:

Offset	Size	Field Name	Forensic Significance
0	16 B	Magic Header String	Mandatory ASCII string: <code>SQLite format 3</code> . Validates authentic file signature.
16	2 B	Page Size	Physical page size (512 to 65,536 bytes, power of 2). Dictates all boundary and offset math.
18	1 B / 1 B	Write / Read Version	File format version (1 = Conventional Rollback Journal, 2 = Write-Ahead Log / WAL mode).
20	1 B	Page Reserved Space	High-Priority Forensic Target: Number of bytes reserved at the footer of each page (0 to 255). Completely invisible to standard SQL queries and never overwritten by B-Tree cell allocations. Prime vector for steganographic data hiding and covert exfiltration.
24	4 B	File Change Counter	Incremented upon every write transaction. Exposes tampering, rollback manipulation, or unauthorized edits.
28	4 B	Database Size in Pages	Logical page count. Exposes <i>EOF Slack</i> (payloads hidden beyond the logical database end).
32	4 B	First Freelist Trunk Page	Pointer to the root page of the decommissioned page trunk list.
36	4 B	Total Freelist Pages	Total count of completely deallocated pages remaining inside the file.
56	4 B	Text Encoding	Database string encoding (1 = UTF-8, 2 = UTF-16le, 3 = UTF-16be).

1.2 B-Tree Page Types

Every page in an SQLite database is structured according to one of 4 fundamental B-Tree page types, indicated by the first byte of the page header:

- **Table Leaf Page (`0x0D`):** Stores application table data payloads and the 64-bit varint RowID. Primary target for cell carving.
- **Table Interior Page (`0x05`):** Routing tree containing RowID keys and child page pointers to locate leaf pages.
- **Index Leaf Page (`0x0A`):** Stores index keys and target RowIDs. Also stores entire rows for `WITHOUT ROWID` tables.
- **Index Interior Page (`0x02`):** Navigation tree pointing to secondary index leaf pages.

1.3 Internal Page Layout & Forensic Slack Zones

An SQLite B-Tree page allocates space from top-down and bottom-up simultaneously, generating distinct physical zones:

Physical Zone	Allocation Vector	Content & Forensic Recovery Significance
Page Header	Page Start (Offset 0 or 100)	Specifies B-Tree page type, first freeblock pointer (offset 1), cell count (offset 3), and cell content start offset (offset 5).
Cell Pointer Array	Downward (toward center)	Array of 2-byte unsigned integers (big-endian) pointing to the starting offset of each active cell.
Unallocated Space	Median Gap	Physical byte gap between the cell pointer array end and the cell content area start. Contains orphaned records from deleted pointers.
Cell Content Area	Upward (from bottom up)	Zone where active cell payloads reside. New cells are allocated upward toward the center.
Freeblocks	Scattered inside Content Area	Deleted cells linked together via a singly-linked list. Preserves intact record payload bodies.
Fragmented Free Slack	Micro-gaps (Header Offset 7)	Fragmented bytes (< 4 bytes) resulting from cell deletion and micro-compaction.
Page Reserved Space	Page Footer (PageSize - Reserved)	Invariable reserved margin at the end of every page, untouched by the B-Tree allocation engine.

2. Carving Algorithms & Low-Level Reverse Engineering

2.1 Freeblock Carving & Overwritten Header Reconstruction

Why: When a row is deleted, SQLite preserves the cell data in place and adds it to a singly-linked list starting at offset 1 of the page header to facilitate rapid space reuse.

The Technical Hurdle: To maintain the linked list, SQLite overwrites the first 4 bytes of the deleted cell with:

```
[ Next Freeblock Offset (2 bytes, uint16_be) ] [ Freeblock Size (2 bytes, uint16_be) ]
```

This destroys the cell's original header (the payload length varint and RowID varint).

The Solution Implemented: SQLite-Carver-Pro isolates the remaining buffer (starting at byte 4) and evaluates candidate serial type sequences (`decode_truncated_record`):

1. Identifies valid candidate *Record Header* sequences.
2. Computes total expected body length: `L_body = Σ serial_type_length(st)`.
3. Enforces boundary invariants: the remaining freeblock buffer must match the declared column lengths exactly, reconstructing deleted data with zero false positives.

2.2 Sliding-Window Unallocated Space Carving

Why: When rows are deleted and pages partially compacted, unreferenced cell fragments remain trapped in the median unallocated gap between the pointer array and the cell content start.

Algorithm Mechanics:

- **Zero-Run Fast Skip:** Unallocated space frequently contains blocks of zeros (padding). The scanner skips consecutive zero bytes instantly, providing a 10x performance increase over raw byte scanning.
- **Dual-Mode Validation:**
 1. *Cell Format:* Parses `payload_size` varint, enforces plausible boundaries `2, 100 ext{ MB}`, reads RowID, and verifies `L_{payload} = ext{header_size} + Σ ext{type_len}`.
 2. *Raw Record Format:* If cell headers are destroyed, parses raw record headers directly, validating text printability ratios and numeric bounds.

2.3 Dropped Table Schema Resurrection (`DROP TABLE`)

Why: Executing `DROP TABLE` deletes the schema entry from `sqlite_master` on Page 1 (moving it to Page 1 unallocated space) and moves all table data pages to the freelist. Conventional forensic tools fail to type freelist rows because table schema metadata is missing.

The Solution:

1. During Page 1 analysis, the carver specifically targets deleted `sqlite_master` records in unallocated space and freeblocks (filtering for `"table"` and `CREATE TABLE` SQL text).
2. Resurrected schemas are dynamically registered into `self.recovered_schemas`.

3. When freelist pages are subsequently carved, a re-matching pass matches orphaned records against the resurrected schema, automatically restoring real column names (e.g., `id, name, surname, code`) instead of generic `col_0, col_1`.

2.4 `WITHOUT ROWID` Architecture Decoding

Technical Specifics: Tables defined with `WITHOUT ROWID` do not use Table B-Trees (`0x0D`). All table records are stored inside Index B-Trees (`0x0A`). There is no RowID in the cell header; primary key columns and data columns are packed together in the record payload.

Implementation: SQLite-Carver-Pro maps the root page of the table in `table_root_map` with `is_without_rowid = True`. Index cells matching this page are decoded directly as primary table records with 1.0 confidence.

2.5 Binary R-Tree Spatial Geometry De-encapsulation

Problem: SQLite R-Tree spatial tables store bounding boxes inside opaque binary BLOBs within shadow tables (`<name>_node`). Standard tools cannot parse these coordinates.

Binary Decoder (`unpack_rtree_leaf_node`):

- Bytes 0–1: `depth` (uint16_be). Leaf nodes strictly require `depth == 0`.
- Bytes 2–3: `n_entries` (uint16_be), number of coordinate entries in the node.
- Each leaf entry consumes $\$8 + 8D\$$ bytes ($D = \text{dimensions}$):
 - 8 bytes: Signed 64-bit integer (int64_be, `rowid`).
 - $\$2D \text{ imes } 4\$$ bytes: 32-bit big-endian IEEE 754 floating-point coordinates (float32_be). For 2D: `minX, maxX, minY, maxY`.

The engine decodes these coordinates into clean virtual table records with genuine column names.

3. Transaction Timeline Diff Engine (WAL & Rollback)

The timeline engine establishes an indisputable sequence of database operations by correlating data across the Write-Ahead Log (WAL) or Rollback Journal.

3.1 WAL Frame Dissection

In WAL mode, writes are appended sequentially to `.db-wal` as fixed-size frames:

```
[ WAL Header (32 B) ] [ Frame 1: Header (24 B) + Page Data ] [ Frame 2: Header (24 B) + Page Data ] ...
```

The 24-byte frame header provides:

- **Page Number (4 B):** Target database page updated by this frame.
- **Commit Counter (4 B):** Non-zero value indicates a committed transaction boundary and database size in pages.
- **Salt-1 / Salt-2 (8 B):** Random sequence values regenerated upon WAL reset to verify frame ordering.
- **Checksum (8 B):** Cumulative verification checksums ensuring frame validity.

3.2 Column-Level Granular Diffs (Before & After)

By comparing successive versions of a row across the database and WAL frames, the engine reconstructs exact mutations:

- `INSERT`: Record created in a WAL frame that did not previously exist.
- `UPDATE`: Row modification with field-by-field before/after diffs (e.g., `balance: $1,500 → $200`).
- `DELETE`: Row present in an earlier transaction but eliminated in subsequent frames.

3.3 WAL Shared Memory Inspection (`.db-shm`)

The `.db-shm` file coordinates concurrent readers and writers. `ShmAnalyzer` inspects the 48-byte header to reveal:

- Checkpoint sequence numbers (`aFrame[0]`).
- Backfill frame watermarks (`aFrame[1]`).
- Active reader lock slots (0–7), proving whether concurrent database connections were open when the file was acquired.

4. Deep Forensic Decoders & Chain of Custody

4.1 Embedded BLOB Microscope

Modern mobile and desktop applications frequently serialize nested data inside SQLite BLOB columns. SQLite-Carver-Pro incorporates 4 recursive inspection decoders:

- **Apple Binary Plist (`bp1ist00`)**: Deserializes dictionaries, arrays, UUIDs, and Cocoa dates without macOS dependencies.
- **Google Protocol Buffers (`protobuf`)**: Dynamic wire-type parser operating without `.proto` definitions, strictly eliminating false positives.
- **Zlib / Deflate Decompression**: Detects zlib headers (`0x78 0x9C` , `0x78 0x01` , `0x78 0xDA`) and inflates compressed streams.
- **File Signature Carving**: Magic-byte detection and disk extraction of embedded PNG, JPEG, GIF, PDF, ZIP, and SQLite files (`--dump-blobs`).

4.2 Normalized Multi-Format Timestamp Engine

The `decode_timestamp` engine normalizes multi-architecture numeric timestamps to UTC:

Timestamp Architecture	Epoch Baseline	Resolution	Validated Date Range
Unix Epoch (Sec)	01/01/1970 00:00:00 UTC	Seconds (10 digits)	Years 1995 to 2038
Unix Milliseconds	01/01/1970 00:00:00 UTC	Milliseconds (13 digits)	Years 2000 to 2040
WebKit / Chrome / Safari	01/01/1601 00:00:00 UTC	Microseconds (17 digits)	Years 2000 to 2040
Windows FILETIME	01/01/1601 00:00:00 UTC	100-nanosecond intervals (18 digits)	Years 2000 to 2040
Apple Cocoa Absolute Time	01/01/2001 00:00:00 UTC	Floating seconds / microsecond integers	Years 2001 to 2040
GPS Time	06/01/1980 00:00:00 UTC	GPS seconds (no leap-second adjustments)	Years 1980 to 2040

4.3 Shannon Entropy & Encryption Assessment

Per-page and global Shannon entropy calculations:

$$H = - \sum (p_i \times \log_2 p_i)$$

Plaintext or partially populated pages range between 1.5 and 5.5. Entropy exceeding **7.95** reliably identifies encrypted database containers (e.g., *SQLCipher*, *wxSQLite3*, or *SEE* using AES-256) or hidden encrypted payloads.

4.4 Cryptographic Chain of Custody & Evidence Hashes

Compliant with ISO/IEC 27037 forensic standards:

- Primary databases and companion files are hashed via SHA-256 and MD5 upon ingestion.
- Every carved record is assigned a deterministic SHA-256 evidence hash: `SHA-256(page_id || offset_in_page || source || matched_table || rowid || values)`.

5. CLI Reference & Standalone Distribution

5.1 Essential CLI Commands

```
# 1. Full carving with interactive HTML dashboard and timeline
sqlite-carver carve evidence.db --wal --html report.html --out evidence.json

# 2. Extract only modified/mutated rows with before/after diffs
sqlite-carver carve evidence.db --mutations-only --html mutations.html

# 3. Raw B-Tree Page Hunter across RAM dumps or disk images
sqlite-carver carve-raw memory_dump.raw --workers 8 --html ram_report.html

# 4. Comprehensive physical storage allocation & slack audit
sqlite-carver storage-breakdown evidence.db

# 5. WAL shared memory (.db-shm) lock diagnostic
sqlite-carver analyze-shm evidence.db-shm

# 6. Shannon entropy calculation & encryption detection
sqlite-carver analyze-encryption evidence.db

# 7. Deep recursive search for tokens, flags, and emails
sqlite-carver search evidence.db "session_token_xyz"
```

5.2 Offline Interactive HTML Dashboard

The standalone HTML dashboard generated with `--html` is an air-gapped web application:

- **Storage Allocation Banner:** Displays exact physical metrics across 6 zones (Active Data, Freeblocks, Unallocated Space, Fragmented Slack, Freelist Pages, and Page Reserved Space).
- **Instant Client-Side Filtering:** Live sub-millisecond search across hundreds of thousands of records.
- **Dedicated WAL Timeline:** Chronological transaction stream with colored mutation badges.
- **Instant Language Toggle:** Seamless English / French switching.

Zero-Dependency Portable Binary

SQLite-Carver-Pro is distributed as a self-contained 64-bit Windows executable (`sqlite-carver-v1.7.0.exe`) requiring zero installation, runtimes, or network connectivity on forensic lab workstations.